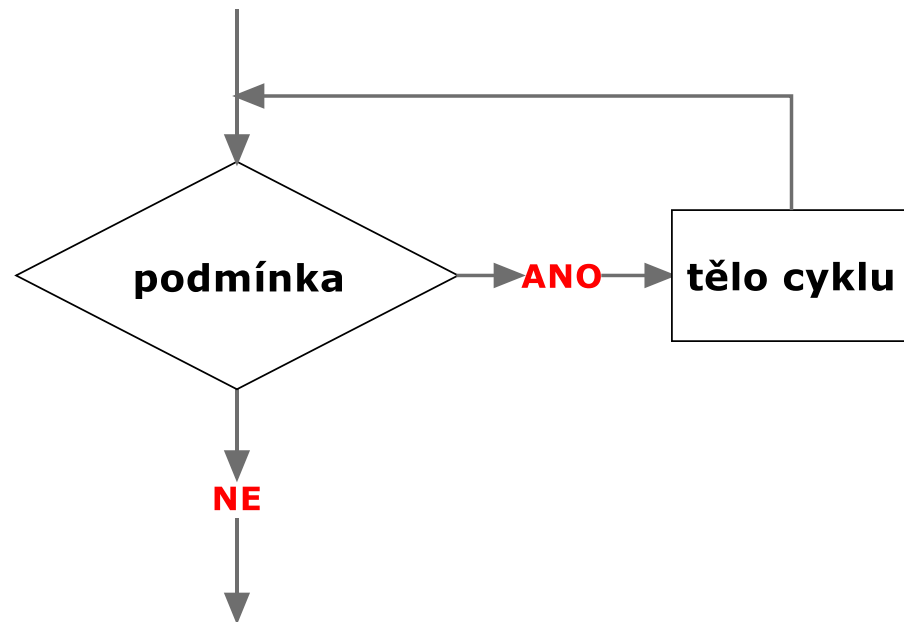
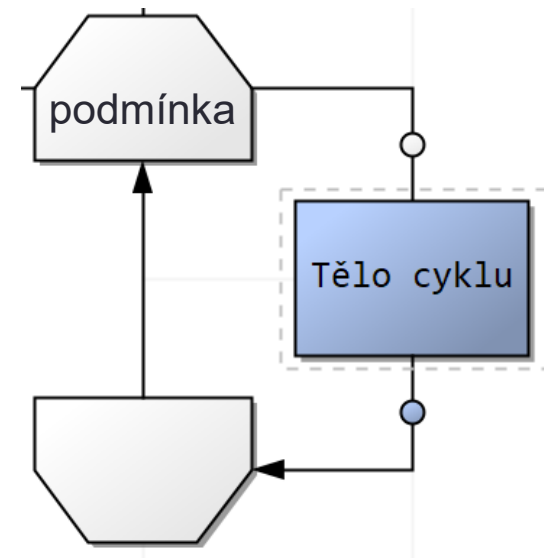


CYKLY

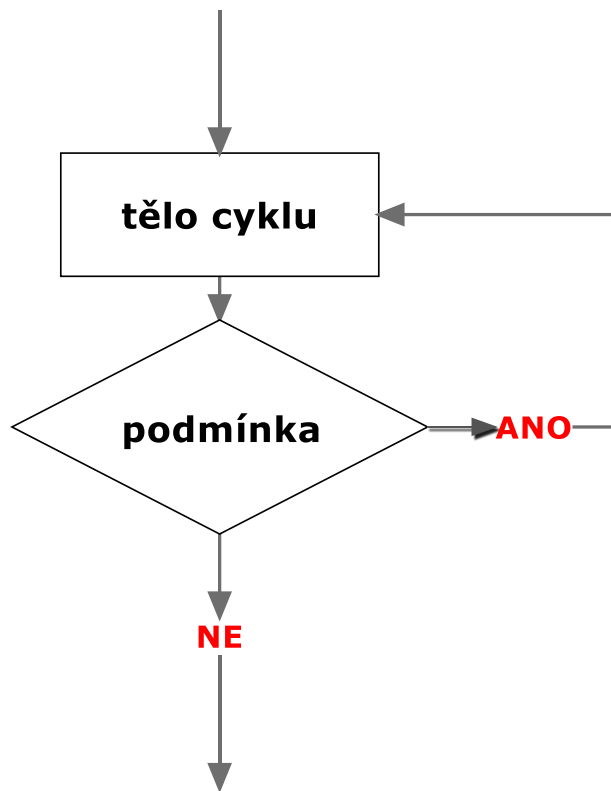
Cyklus s podmínkou na začátku



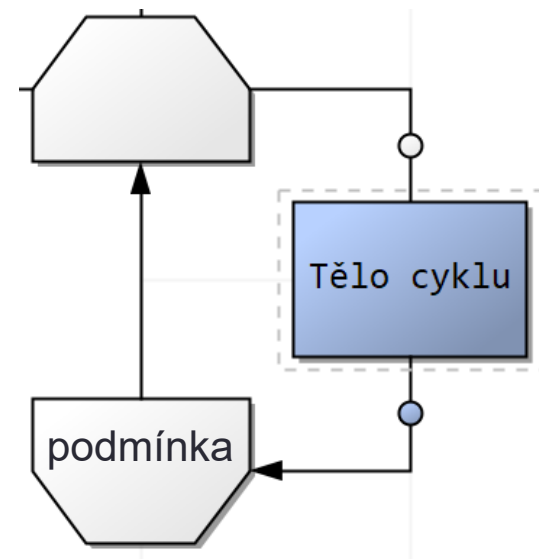
v PS Diagramu



Cyklus s podmínkou na konci



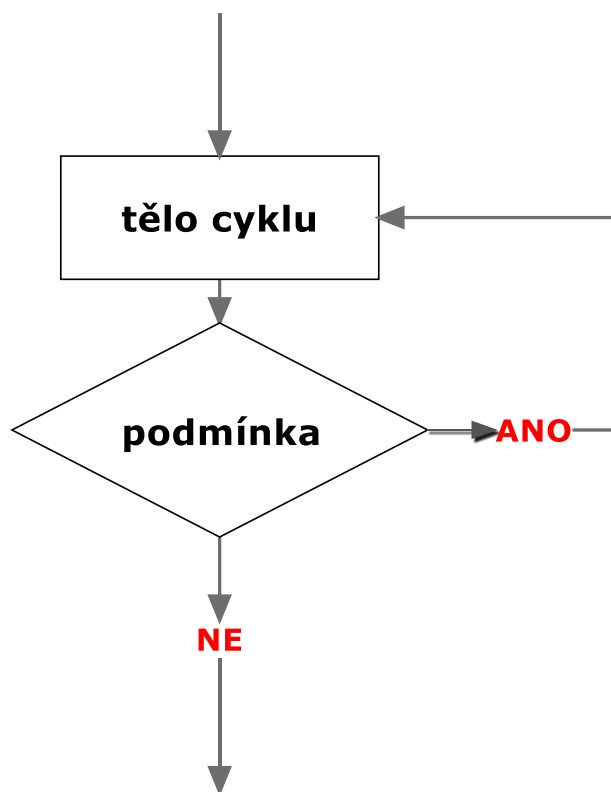
v PS diagramu



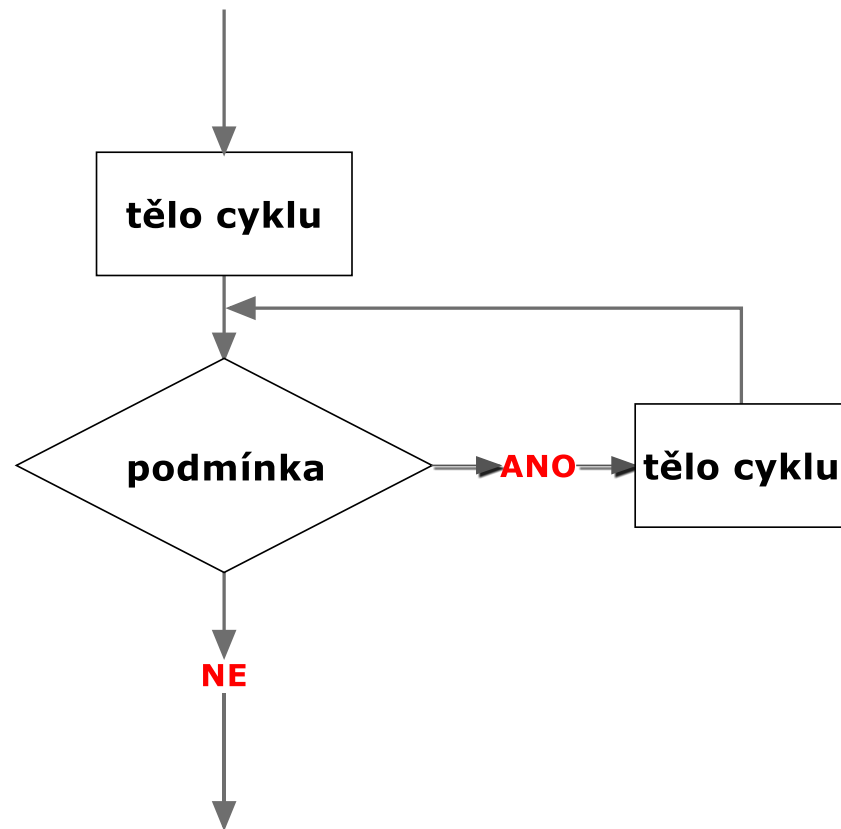
tělo cyklu se provede minimálně jednou

Převod cyklu s podmínkou na konci na cyklus s podmínkou na začátku lze vždy

s podmínkou na konci

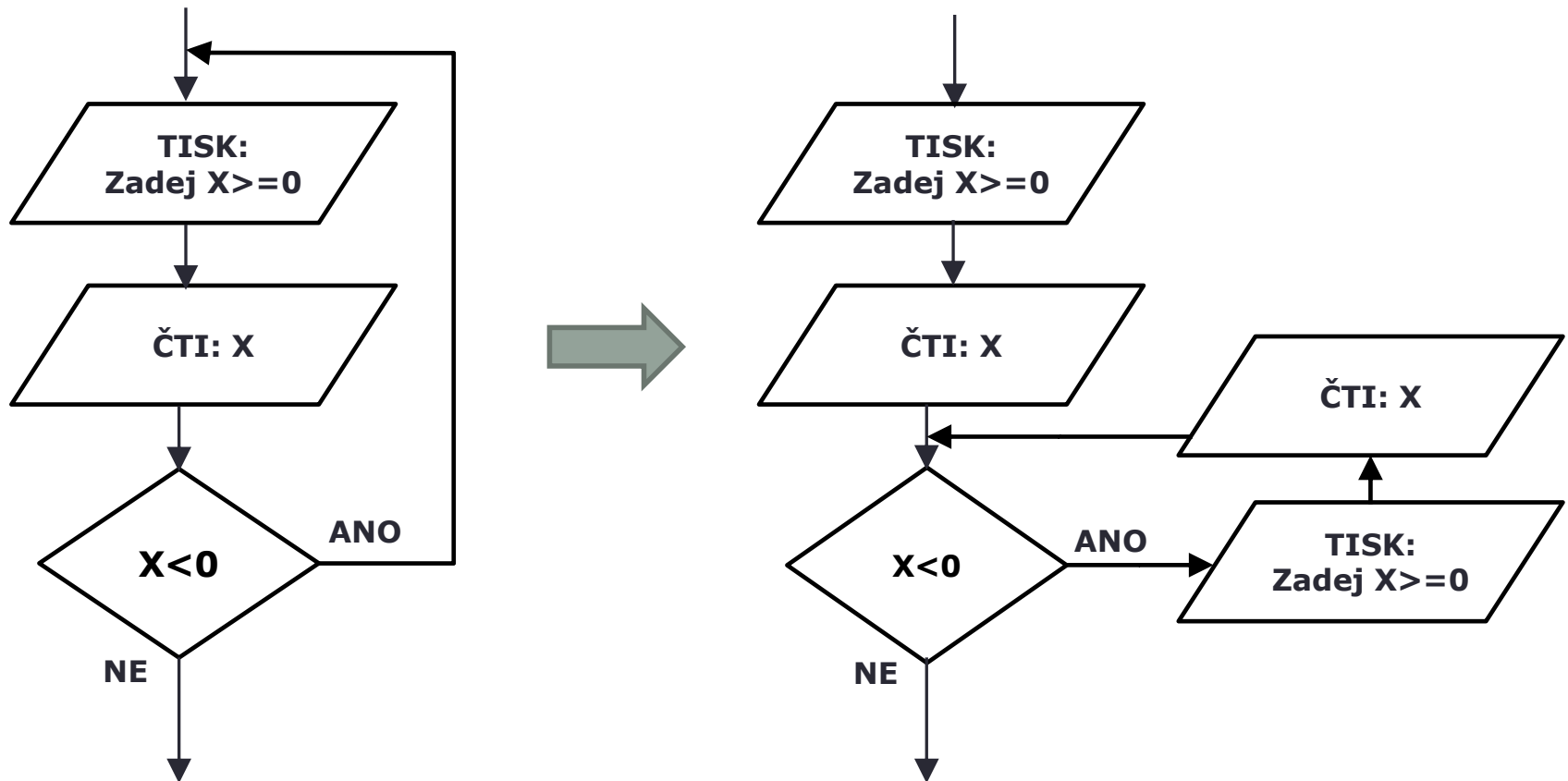


s podmínkou na začátku



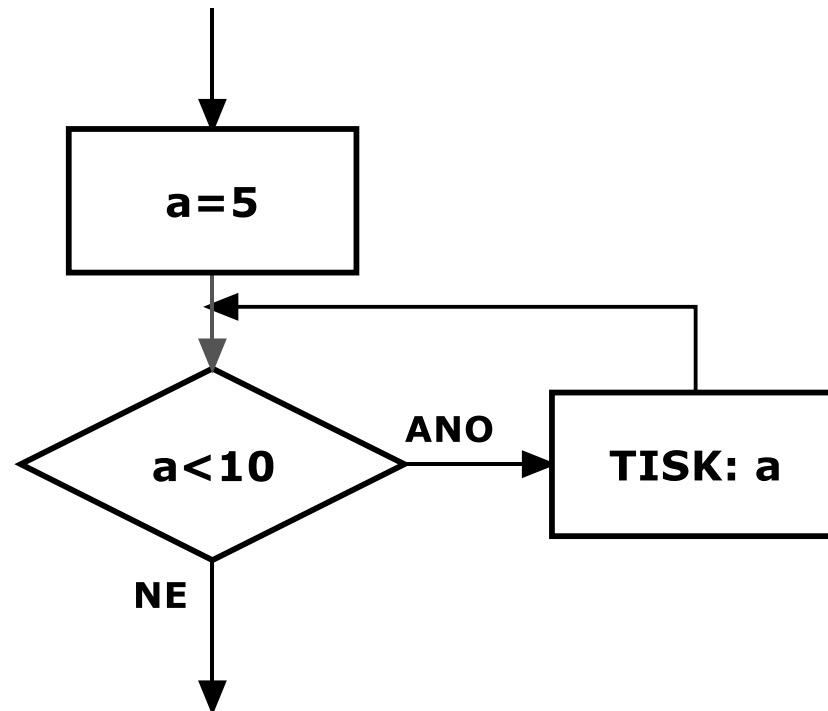
Obecně - převod cyklu s podmínkou na začátku na cyklus s podmínkou na konci nelze.

Cyklus s podmínkou na konci - převod



Pozor na zacyklení (nekonečný cyklus)

- v těle cyklu musí být operací, která má vliv na podmínku. Pokud tam taková operace není a podmínka cyklu platí, bude platit „do konce věků“...



Algoritmus pro zatlučení hřebíku

- tlučeme tak dlouho, dokud není hřebík zatlučený = cyklus s podmínkou

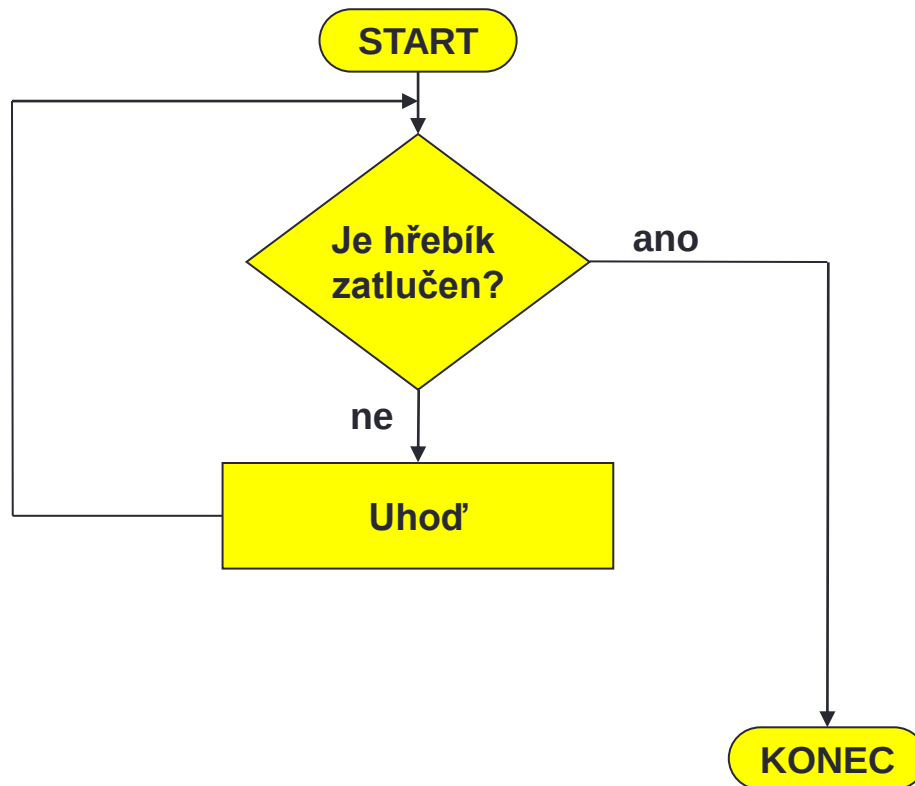


operace: **Uhod'** = jeden úder kladiva do hřebíku

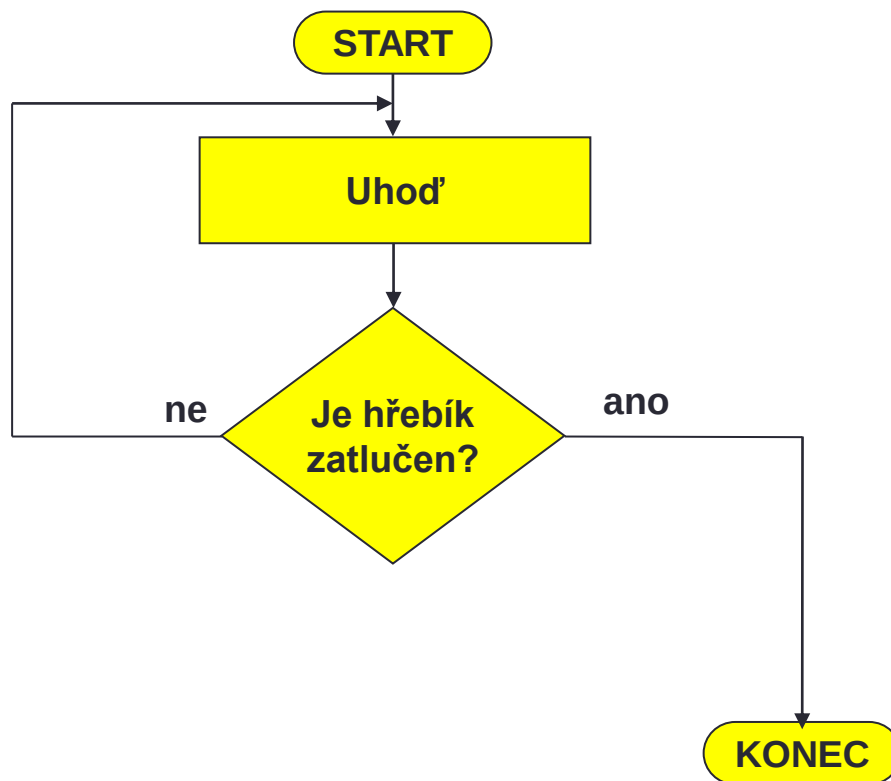
podmínka: **Je hřebík zatlučen?** = platí, pokud
hlavička hřebíku je v úrovni dřeva

pozn. uvažujeme ideální svět, kde se hřebíky neohýbají...

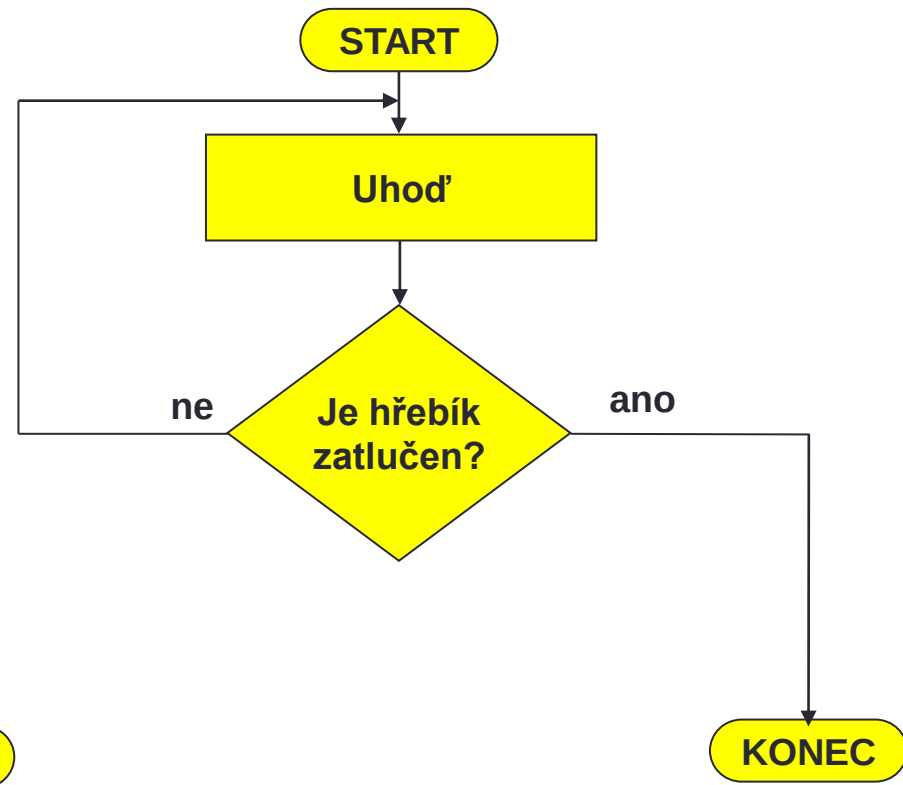
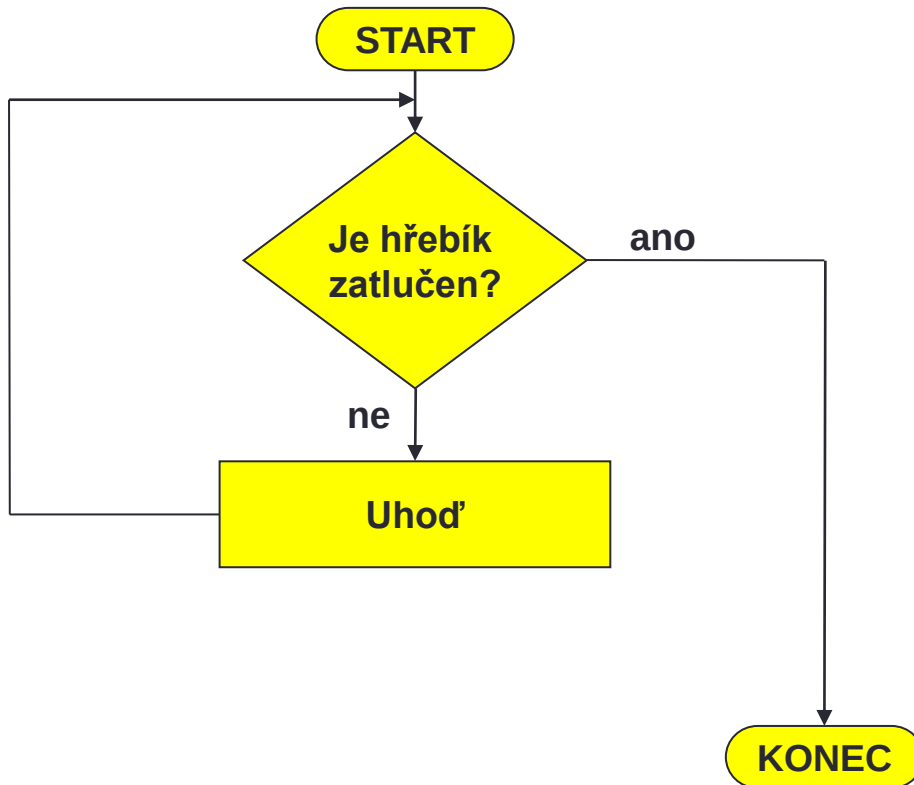
Cyklus s podmínkou na začátku



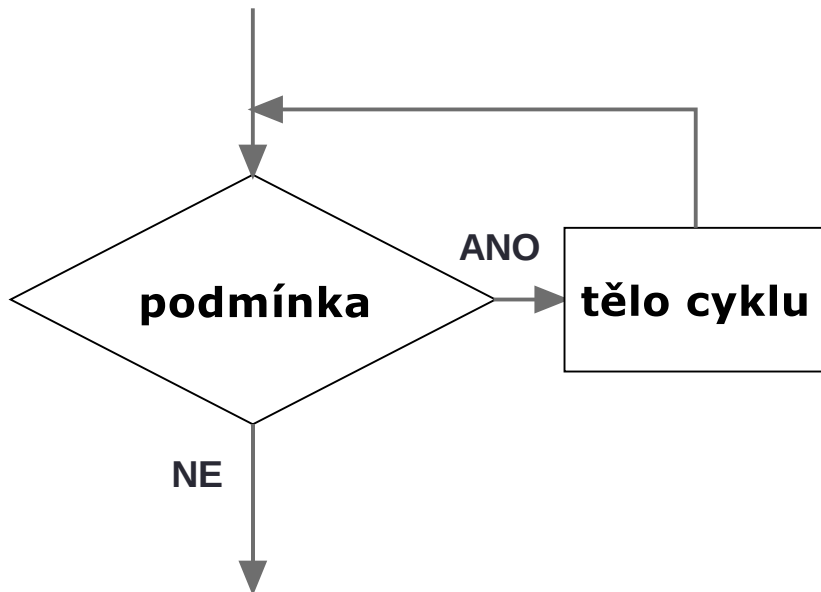
Cyklus s podmínkou na konci



Které řešení je lepší?

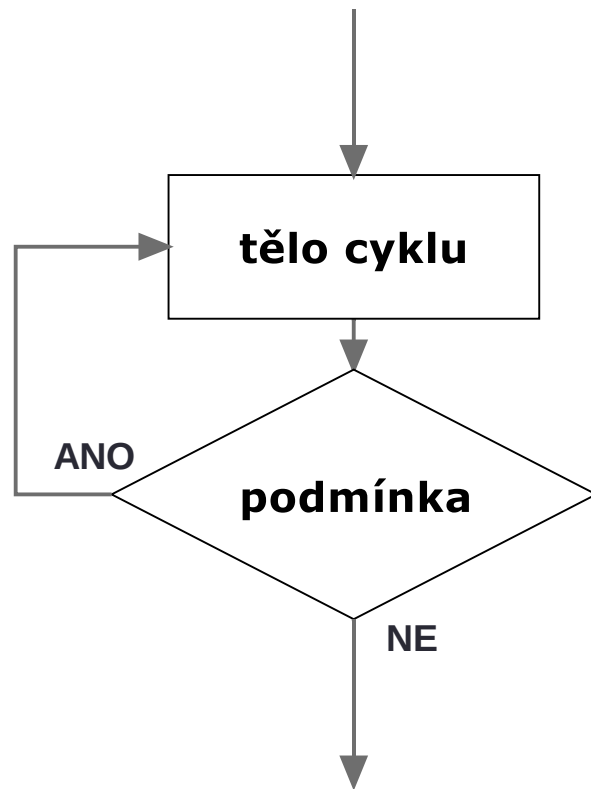


Cyklus s podmínkou na začátku - syntaxe



```
while (podmínka)
{
    příkaz1;
    příkaz2;
    ...
}
```

Cyklus s podmínkou na konci - syntaxe



```
do
{
    příkaz1;
    příkaz2;
    ...
}
while (podmínka);
```

Poznámka k podmínkám – ve větvení a cyklu

0 = FALSE, cokoliv jiného = TRUE

```
int a;  
while (1) {  
    printf("Zadej cislo 5: ");  
    scanf("%d",&a);  
    if (a == 5)  
        break;  
}
```

K booleovským hodnotám v jazyce C více viz
<https://www.sallyx.org/sally/c/stdbool.php>
<https://www.sallyx.org/sally/c/c05.php#bool>

Zadání

Vytvořte vývojový diagram a následně program podle něj, který bude číst ze vstupu (klávesnice) celá čísla, dokud uživatel nezadá 0 (nulu). Poté program vypíše:

1. kolik uživatel zadal celkem čísel bez 0
2. kolik zadaných čísel bylo z intervalu $<3;10>$

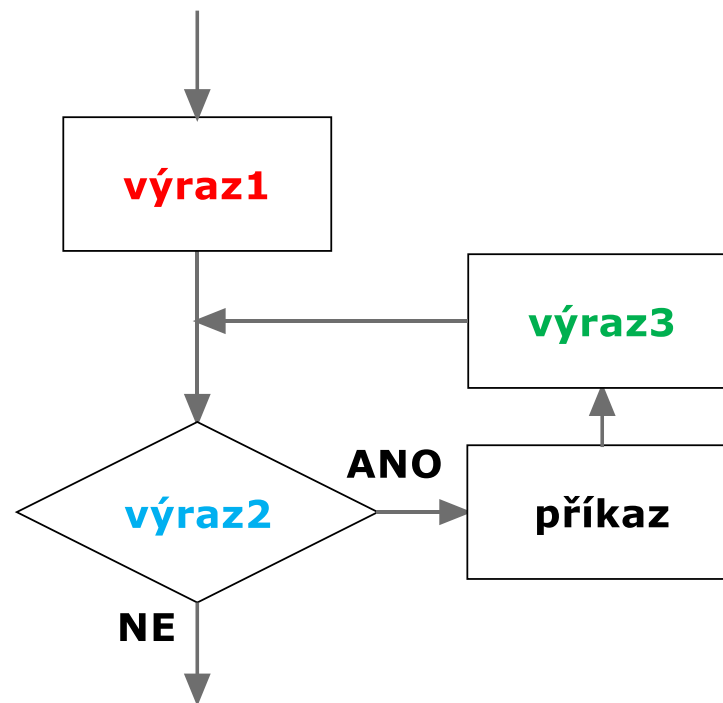
CYKLUS FOR

Cyklus FOR

for(**výraz1**;**výraz2**;**výraz3**) příkaz;

chování cyklu FOR je ekvivalentní cyklu WHILE

```
výraz1;  
while(výraz2)  
{  
    příkaz;  
    výraz3;  
}
```



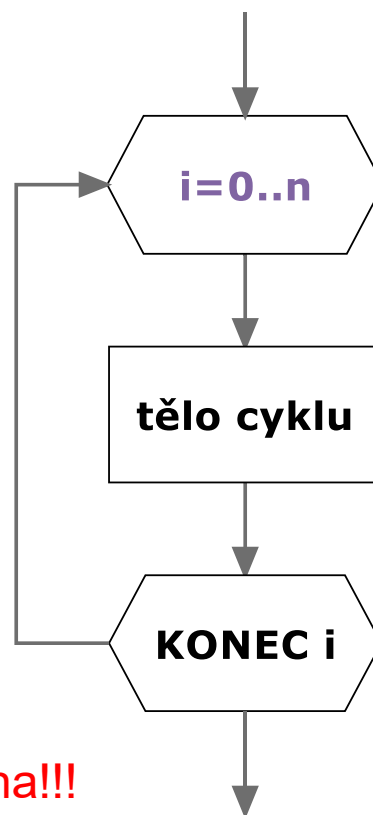
Cyklus FOR - inkrementace

- pokud proměnná cyklu zvyšuje nebo snižuje svoji hodnotu o jedničku, odpovídá to cyklu s pevným počtem opakování

```
for(i=0;i<=n;i++) {  
    tělo cyklu  
}
```

//ekvivalentní s

```
for(i=0;i<=n;i=i+1) {  
    tělo cyklu  
}
```



Proměnná „i“ musí být deklarována!!!

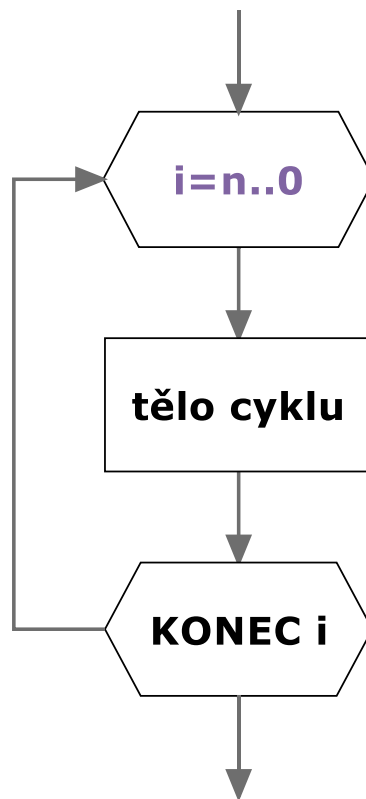
Cyklus FOR - dekrementace

- pokud proměnná cyklu zvyšuje nebo snižuje svoji hodnotu o jedničku, odpovídá to cyklu s pevným počtem opakování

```
for(i=n; i>=0; i--) {  
    tělo cyklu  
}
```

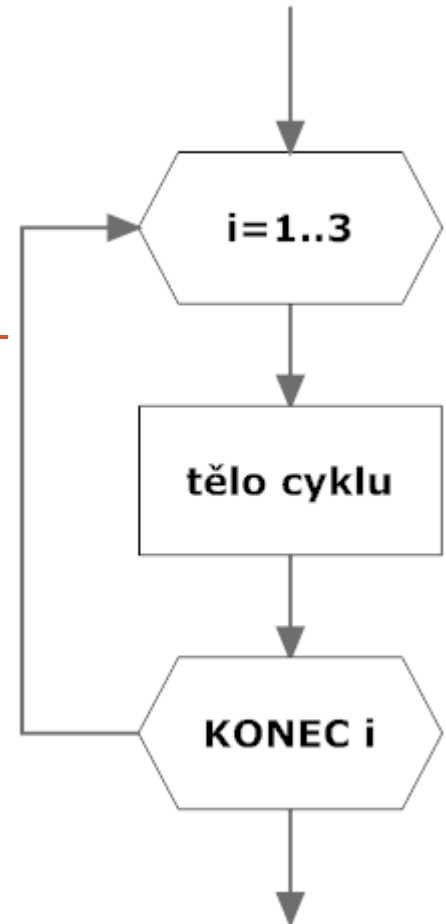
//ekvivalentní s

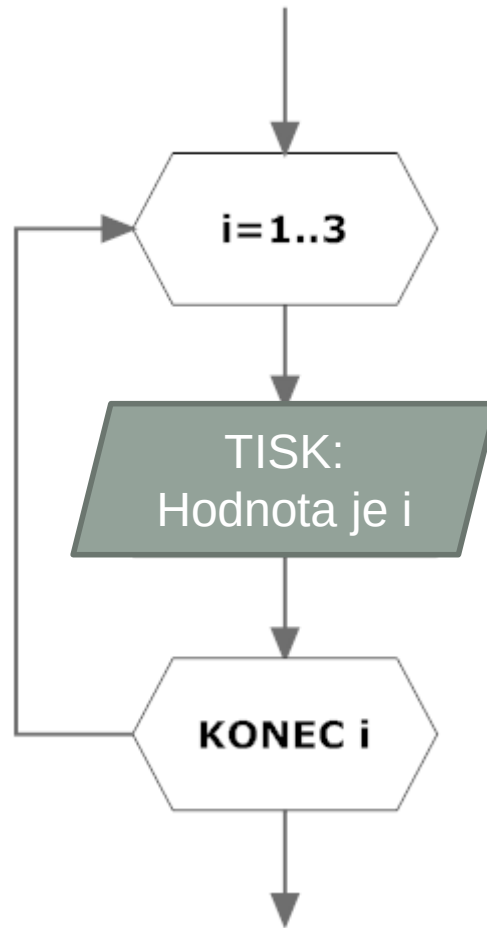
```
for(i=n; i>=0; i=i-1) {  
    tělo cyklu  
}
```



CYKLUS

cyklus s pevným počtem opakování





```
int i;  
for(i=1;i<=3;i++) printf("Hodnota je %d\n",i);
```

Algoritmus – součet přirozených čísel $<0;X>$

Vytvořte vývojový diagram pro algoritmus, který načte přirozené číslo X a vypíše součet neklesající posloupnosti přirozených čísel z intervalu $<0;X>$

- pozn.:
 - ošetřete vstupní hodnotu, aby $X \geq 0$. Pokud nebude splněno, po uživateli opakovaně vyžadujte zadání hodnoty, dokud nezadá použitelnou hodnotu.

X	Provedená operace	Výstup
5	$0+1+2+3+4+5$	15

Jaký bude výpis tohoto programu?

```
int main()
{
    int n,s,i;
    printf("Zadej prirodzene cislo: ");
    scanf("%d",&n); //předpokládáme zadání čísla 5

    s = 0;
    for(i=1;i<=n;i++) s=s+i;
    printf("Soucet je %d",s);
}
```

jak bude algoritmus
pracovat, zadáme-li
záporné n ?

Jaký bude výpis tohoto programu?

```
int i,x;
x = 1;
for(i=1;i<5 && i>=x;i++)
{
    x = x*++i;
    printf("%d ",x);
}
putchar('\n'); // výpis jednoho znaku
for(i=0;i<2;i=i+2)
{
    printf("%d ",i);
}
putchar('\n'); // výpis jednoho znaku
printf("%d",i+1);
```

Vytvořte algoritmus, který vypočítá n-tou mocninu čísla X (X^n)

- můžete použít pouze základní aritmetické operace

+, -, *, /, %

a funkci `abs()` pro výpočet absolutní hodnoty (nutno mít direktivu `#include <stdlib.h>`)

$X^n = X * X * X * \dots * X$ (n-krát násobení X pro $n > 0$)

$$X^n = \frac{1}{X * X * X * \dots * X} \quad (\text{pro } n < 0)$$

$$X^n = 1 \quad (\text{pro } n = 0)$$

Vytvořte algoritmus, který bude počítat faktoriál ze zadaného celého čísla $R \geq 0$

$$R! = R * (R-1)!$$

$$R! = R * (R-1) * \dots * 2 * 1$$

$$R! = 1 * 2 * \dots * (R-1) * R$$

$$0! = 1$$

Na vstupu ošetřete, zda uživatel zadal nezáporné R .
Pokud zadal záporné R , znovu čtěte R tak dlouho,
dokud není zadáno číslo správně, tedy ≥ 0 .